EECE 260: Engineering Programming & Application

Final Project Assignment

Spring 2026 · Howard University

Instructor: Dr. Su Yan

Overview

The final project is an individual assignment where you apply the programming and numerical tools learned this semester to a real-world modeling problem. The theme this year is **mathematical modeling of dynamic systems**, including populations, diseases, conflict, cooperation, and chaos. You will discover that a handful of simple tools (`for` loops, functions, NumPy arrays, Forward Euler, Matplotlib) are enough to simulate remarkably rich phenomena.

Choose one of the nine projects below, implement it in Python, and submit a short report with your code. The projects span a range of difficulties (⋆⋆ to ⋆⋆⋆⋆⋆) so you can pick something at your comfort level or stretch yourself for bonus credit.

General Requirements

1. All code must be original, well-commented, and runnable with `python3 main.py` (or a clearly-named entry script).
2. Use only the packages we covered this semester: `numpy`, `matplotlib`, and the Python standard library. If you feel you need `scipy.integrate`, you may use it *only* as a reference to verify your own solver, not as the primary solver.
3. Figures: PDF or PNG with axis labels, titles, and legends. Each figure must be referenced and discussed in your report.
4. Report: **2–3 pages** (excluding figures), covering problem formulation, method, results, and discussion.
5. Submit all files (code + figures + report) as a single `.zip` via Canvas by the due date.

Grading Rubric (Common to All Projects)

Component	Weight
Correctness of implementation	35%
Completion of required tasks	20%
Accuracy / convergence / sensitivity analysis	15%
Code quality & documentation	10%
Figures & visualization	10%
Written report (clarity & discussion)	10%

Each project also lists project-specific grading focus items that I will pay special attention to. **Difficulty bonus:** Projects of ⋆⋆⋆⋆ and ⋆⋆⋆⋆⋆ difficulty earn up to **5** and **8** bonus points, respectively, when implemented well.

Project Summary Table

#	Project	Core Method	Domain	Difficulty
1	Logistic Population Growth	Forward Euler (1-D ODE)	Ecology / Biology	⋆⋆
2	Predator–Prey (Lotka–Volterra)	Vector ODE + Newton's method	Ecology	⋆⋆⋆
3	SIR Epidemic	Vector ODE (3-D)	Epidemiology	⋆⋆⋆
4	Newton's Cooling + Inverse	ODE + Root-finding	Physics / Forensics	⋆⋆
5	Lanchester Warfare	Coupled ODE, two model forms	Military history	⋆⋆⋆
6	Prisoner's Dilemma Tournament	Functions as arguments, loops	Game theory	⋆⋆⋆
7	Hawk–Dove Evolutionary Game	Replicator ODE + Newton	Evolutionary biology	⋆⋆⋆⋆
8	Cournot Duopoly	Discrete dynamical system	Economics	⋆⋆⋆⋆
9	Lorenz Attractor	Vector ODE + RK4, 3-D visualisation	Nonlinear dynamics	⋆⋆⋆⋆⋆

How to choose: All nine are self-contained, and you only need to do *one*. If you want a safe, well-paced project, start with 1 or 4. If you want something different from the usual numerical problem set, try 6 (game theory, no ODEs at all). If you want bonus credit and a great portfolio piece, try 7, 8, or 9. Projects 5 and 6 are good for history/social-science-minded students; 7 and 8 for those interested in economics or biology; 9 for anyone drawn to chaos and beautiful visualisations.

A note on scope: Bonus tasks are genuinely optional; a clean and correct implementation of the *required* tasks earns full base credit. If in doubt, do the required tasks well rather than the bonus tasks poorly.

Project Descriptions

Project 1: Logistic Population Growth ⋆⋆

Background. The simplest population model is exponential: $\dot{P} = rP$. But in reality no population grows forever because food, space, or disease eventually limit it. The *logistic* model introduces a *carrying capacity* K, giving an S-shaped (sigmoidal) growth curve that matches bacteria in a petri dish, deer on an island, or users of a new app surprisingly well.

Mathematical Model.

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right), \qquad P(0) = P_0.$$

Parameters: $r = 0.5/\text{yr}$ (intrinsic per-capita growth rate when the population is small and unconstrained), $K = 1000$ (carrying capacity, the maximum population the environment can sustain), $P_0 = 50$ (initial population); simulate over $t \in [0, 30]\,\text{yr}$. The analytical solution is $P(t) = K\big/\left(1 + \frac{K-P_0}{P_0}\,e^{-rt}\right)$, which can be used to verify your numerical solver, but do *not* use it as the simulation.

Getting Started.

1. Review Lesson 20 (Forward Euler) and Lesson 15 (Matplotlib basics).
2. Write a *generic* Forward-Euler solver: `forward_euler(f, y0, t_span, dt)` that takes any rate function `f(t, y)` and returns arrays `t` and `y`. You will reuse this solver in later problems.
3. Write the logistic rate function: `def logistic_rate(t, P): return r*P*(1 - P/K)`.
4. Call the solver and plot $P(t)$. Overlay the analytical solution to check.

Tasks.

1. Implement a generic Forward-Euler solver and apply it to the logistic equation with $\Delta t = 0.1$. Plot $P(t)$ and overlay the analytical solution.
2. **Compare with exponential growth:** On the same axes, plot logistic vs. pure exponential ($\dot{P} = rP$). Discuss when they agree and when they diverge.
3. **Initial-condition sensitivity:** Plot five trajectories from $P_0 \in \{1, 10, 100, 500, 1500\}$ on the same figure. What happens when $P_0 > K$?
4. **Parameter sensitivity:** A 3×3 grid of subplots for $(r, K) \in \{0.2, 0.5, 1.0\} \times \{500, 1000, 2000\}$.
5. **Convergence study:** Compute L^∞ error between numerical and analytical solutions for $\Delta t \in \{1.0, 0.5, 0.2, 0.1, 0.05, 0.01\}$. Log-log plot of error vs. Δt; verify the $O(\Delta t)$ slope of Forward Euler.
6. **Seasonal extension (optional):** Let the carrying capacity oscillate, $K(t) = K_0 + A\sin(2\pi t/T_s)$ with $T_s = 5\,\text{yr}$, $A = 300$. Describe the resulting dynamics.

Deliverables. `euler_solver.py`, `logistic.py` (main script); 5–6 PDF figures; report.

Project-Specific Grading Focus.

- Your Euler solver must be *generic* (reusable for any ODE), not hard-coded to this problem. I will inspect the function signature.
- The convergence plot must show the correct slope (≈ 1) on log-log axes.
- The report must discuss *why* logistic growth saturates and what happens physically when $P_0 > K$.

Project 2: Lotka–Volterra Predator–Prey Dynamics ⋆⋆⋆

Background. From 1845–1935, the Hudson's Bay Company recorded the number of lynx and snowshoe hare pelts traded in Canada. The counts oscillated with a strikingly regular ∼10-year period. The Lotka–Volterra equations (1925–1926) are the canonical explanation: prey grow, predators eat prey, predators starve without prey. This model underlies modern ecology, but also epidemiology and even economics.

Mathematical Model. Let $R(t)$ = prey population, $F(t)$ = predator population:

$$\frac{dR}{dt} = \alpha R - \beta RF, \qquad \frac{dF}{dt} = \delta RF - \gamma F.$$

Parameters:

- $\alpha = 1.0/\text{yr}$: prey's intrinsic birth rate (per-capita reproduction when predators are absent).
- $\beta = 0.02/(\text{predator} \cdot \text{yr})$: predation rate coefficient (how often a predator–prey encounter kills a prey per unit time).
- $\delta = 0.01/(\text{prey} \cdot \text{yr})$: predator's reproduction efficiency (fraction of consumed prey converted into new predators).
- $\gamma = 0.5/\text{yr}$: predator's natural death rate (starvation rate when prey are absent).

Initial conditions: $R(0) = 50$, $F(0) = 20$; simulate over $t \in [0, 60]\,\text{yr}$. The non-trivial steady state ("co-existence" equilibrium) is $(R^*, F^*) = (\gamma/\delta,\ \alpha/\beta)$.

Getting Started.

1. Generalize your Forward-Euler solver to accept a *vector* state $\mathbf{y} = [R, F]$ and a vector rate function $\mathbf{f}(t, \mathbf{y})$. Internally, just update each component.
2. Write the LV rate function returning a NumPy array of two values.
3. Solve and plot $R(t)$ and $F(t)$ on the same time axis. You should see the classic out-of-phase oscillations.
4. Plot F vs. R (*phase portrait*); the trajectory should form a closed loop in theory.

Tasks.

1. Extend the Forward-Euler solver to handle vector-valued ODEs.
2. Simulate with $\Delta t = 0.01$; plot $R(t)$ and $F(t)$.
3. **Phase portrait:** Overlay trajectories for 3 distinct initial conditions on the R–F plane. Mark the equilibrium point.
4. **Equilibrium verification:** Compute (R^*, F^*) analytically. Then use Newton's method (Lesson 10) on the 2-D system $\mathbf{f}(R, F) = \mathbf{0}$ to re-derive it numerically. Report the number of iterations.
5. **Period study:** Small-amplitude oscillations have theoretical period $T \approx 2\pi/\sqrt{\alpha\gamma}$. Numerically estimate the period (peak-to-peak) and compare. *Note:* the formula is the linearisation about (R^*, F^*), so it is only accurate for trajectories very close to equilibrium. The baseline orbit is not small-amplitude (it starts at $F(0) = 20$, far from $F^* = 50$), so your measured period will typically be 5–15% larger than the formula predicts. Report both values and the relative difference rather than treating the discrepancy as a bug.
6. **Numerical drift:** Forward Euler is dissipative and the closed loop will *spiral outward*. Quantify the drift by plotting the conserved quantity $H(R, F) = \delta R - \gamma \ln R + \beta F - \alpha \ln F$ over time. (Bonus: Implement RK4 and show the drift is drastically reduced.)
7. **Interpretation:** Why does adding a predator *decrease* the equilibrium prey population only through γ (predator death rate), not through α or β?

Deliverables. `euler_vector.py`, `lv_dynamics.py`; 4–5 PDF figures (time series, phase portrait, $H(t)$ drift, period); report with a phase-portrait interpretation.

Project-Specific Grading Focus.

- The phase portrait must show at least 3 trajectories with the equilibrium clearly marked.
- The equilibrium must be verified with Newton's method (this ties back to Lesson 10).
- The report must explain the numerical drift you observe (this is the most commonly misunderstood aspect).

Project 3: SIR Epidemic Model ⋆⋆⋆

Background. When COVID-19 spread globally, the SIR model (Kermack–McKendrick, 1927) was on every news broadcast. It divides the population into three compartments: Susceptible, Infected, Recovered. Despite its simplicity, it explains peak infection timing, herd immunity thresholds, and the effect of vaccination strategies.

Mathematical Model. Let N = total population (constant). Then

$$\frac{dS}{dt} = -\frac{\beta SI}{N}, \qquad \frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I, \qquad \frac{dR}{dt} = \gamma I.$$

The *basic reproduction number* is $\mathcal{R}_0 = \beta/\gamma$: the expected number of secondary infections from one infected person in a fully susceptible population. If $\mathcal{R}_0 > 1$, the epidemic grows; if $\mathcal{R}_0 < 1$, it dies out. (Note the distinction: $R(t)$ is the *recovered* compartment; $\mathcal{R}_0$ is the *basic reproduction number*. We use $\mathcal{R}_0$ throughout to avoid confusion with the initial recovered population $R(0)$.)

Parameters: $N = 10{,}000$ (total population, fixed), $S(0) = 9{,}999$, $I(0) = 1$, $R(0) = 0$ (initial susceptible / infected / recovered), $\beta = 0.5$/day (transmission rate, effective contact rate multiplied by the probability of infection per contact), $\gamma = 0.1$/day (recovery rate, the reciprocal of the mean infectious period, so infected individuals recover on average in $1/\gamma = 10$ days), giving $\mathcal{R}_0 = \beta/\gamma = 5$; simulate over $t \in [0, 150]$ days.

Getting Started.

1. Reuse your vector Forward-Euler solver from Project 2.
2. State is now 3-dimensional: $\mathbf{y} = [S, I, R]$.
3. Solve, plot all three curves on one figure; confirm S decreases, R increases, and I rises then falls.
4. Compute peak I and the time at which it occurs.

Tasks.

1. Solve the SIR system; plot $S(t), I(t), R(t)$ on one figure.
2. **Conservation check:** Plot $S(t) + I(t) + R(t) - N$ vs. t. The error should be tiny ($\sim 10^{-10}$). This validates your integrator.
3. **Peak analysis:** Numerically find the time t_{peak} and value I_{peak}.
4. **Sweep $\mathcal{R}_0$:** Fix γ, vary β so that $\mathcal{R}_0 \in \{0.5, 1, 1.5, 2, 3, 5, 10\}$. For each, plot I_{peak}/N as a function of $\mathcal{R}_0$. What happens at $\mathcal{R}_0 = 1$?
5. **Herd immunity:** The theoretical threshold is $S^*/N = 1/\mathcal{R}_0$; once S falls below this, I can only decrease. Verify numerically: for the baseline $\mathcal{R}_0 = 5$, plot $I(t)$ alongside the line $S(t)/N = 0.2$; the crossing should coincide with the peak.
6. **Vaccination:** Start with $R(0) = 0.4 \cdot N$ (40% pre-vaccinated) and $S(0) = 0.6 \cdot N - 1$. Compare the peak with the unvaccinated scenario.

7. **Final size:** Plot $S(\infty)/N$ (the fraction who escape infection) vs. $\mathcal{R}_0$. This is called the "final-size relation" and is a key public-health quantity.

Deliverables. `sir.py`; 5–6 PDF figures (time series, conservation, peak vs. R_0, herd immunity, vaccination, final size); report.

Project-Specific Grading Focus.

- The conservation check must be presented; it is a sanity test every numerical ODE paper uses.
- The herd-immunity visualisation must explicitly show that the peak occurs when $S/N = 1/\mathcal{R}_0$.
- The report must clearly define $\mathcal{R}_0$ and discuss the public-health meaning of your final-size plot.

Project 4: Newton's Law of Cooling & Inverse Parameter Estimation ⋆⋆

Background. When a hot object is placed in a cooler environment, its temperature approaches ambient exponentially. This is how coroners estimate time-of-death from body temperature, and how you decide when your coffee is drinkable. The problem has two sides: the *forward problem* (given k, compute $T(t)$) and the *inverse problem* (given a measured $T(t)$, estimate k); the latter ties the root-finding tools from Lesson 10 back into ODEs.

Mathematical Model.

$$\frac{dT}{dt} = -k\big(T(t) - T_{\text{env}}\big), \qquad T(0) = T_0.$$

Analytical solution: $T(t) = T_{\text{env}} + (T_0 - T_{\text{env}})e^{-kt}$.

Parameters: $T_0 = 90°\text{C}$ (initial temperature of the object), $T_{\text{env}} = 22°\text{C}$ (surrounding ambient temperature, assumed constant), $k = 0.05\,\text{min}^{-1}$ (cooling rate constant, encapsulating surface area, heat capacity, and the convective/conductive heat-transfer coefficient, so a larger k means faster approach to ambient); simulate over $t \in [0, 60]\,\text{min}$.

Getting Started.

1. **Forward problem:** Reuse your Forward-Euler solver; solve for $T(t)$ and overlay the analytical curve.
2. **Inverse problem:** Suppose an independent experiment records a cooling curve for a different object, and the sensor measures $T(10\,\text{min}) = 63.3°\text{C}$ with $T_0 = 90°\text{C}$ and $T_{\text{env}} = 22°\text{C}$ (same as baseline). Estimate k. Formulate as a root-finding problem: $g(k) = T_{\text{numerical}}(10; k) - 63.3 = 0$, then solve with bisection *and* Newton's method (Lesson 10). You should recover a value close to the baseline $k = 0.05$, which validates the inverse procedure.

Tasks.

1. Solve the forward problem; overlay analytical solution; report maximum error.
2. **Convergence:** Compute the error for $\Delta t \in \{5, 2, 1, 0.5, 0.1, 0.05\}\,\text{min}$. Log-log plot.
3. **Inverse problem with bisection:** Given $T(10) = 63.3°\text{C}$, find k using bisection over $[10^{-4}, 1]\,\text{min}^{-1}$.
4. **Inverse problem with Newton:** Same problem, using Newton's method with a numerical derivative (dg/dk via finite difference). Compare iteration count with bisection.

5. **Forensic application:** A body is discovered at $t = 0$ with $T_0 = 34°\text{C}$ (ambient = 22°C). Normal body temperature is 37°C and an empirically calibrated cooling rate for a human body (Glaister equation approximation) is $k = 0.0015\,\text{min}^{-1}$ (equivalently $0.09\,\text{hr}^{-1}$). *Estimate the time of death* (a negative t) by solving the inverse problem: at what $t^* < 0$ was $T(t^*) = 37°\text{C}$? Report your answer in hours for readability.
6. **Multi-point fit (bonus):** Given 5 noisy measurements of $T(t_i)$, fit k by least-squares: minimise $\sum_i (T_{\text{model}}(t_i; k) - T_i)^2$ using either a grid search or gradient descent. Report the estimated k and comment on uncertainty.

Deliverables. `cooling.py`; 3–4 PDF figures; report with the forensic estimate and its reasoning.

Project-Specific Grading Focus.

- The inverse problem must be framed as a root-finding problem on a scalar function $g(k)$.
- Bisection and Newton's method must be compared (iteration counts, accuracy).
- The forensic calculation must be dimensionally and logically correct; carelessness here loses points.

Project 5: Lanchester Models of Warfare ⋆⋆⋆

Background. In 1916, engineer Frederick Lanchester published two simple ODE models to explain aerial combat. He showed that with modern ranged weapons, the side with *more* troops has a quadratic advantage, the famous "principle of concentration of force" that drove 20th-century military doctrine. Ancient close-combat warfare, by contrast, obeys a linear law. These are striking consequences of two small differences in the equations.

Mathematical Model. Let $R(t)$, $B(t)$ be the troop counts of the Red and Blue forces. Let a be Red's fighting effectiveness (Blue troops killed per Red soldier per unit time) and b be Blue's (Red troops killed per Blue soldier per unit time).

$$\begin{aligned} \text{Square law (modern, ranged):} \quad & \dot{R} = -bB, \qquad \dot{B} = -aR \\ \text{Linear law (ancient, melee):} \quad & \dot{R} = -b\,RB/R_0, \qquad \dot{B} = -a\,RB/R_0 \end{aligned}$$

Parameters: $R_0 = 1000$ and $B_0 = 800$ (initial troop strengths of Red and Blue); $a = 0.15$ per unit time (Red's per-soldier offensive lethality: the rate at which each Red soldier kills Blue troops, combining Red's weapon accuracy and rate of fire with Blue's armour or cover, since a tougher target reduces the effective kill rate); $b = 0.20$ per unit time (Blue's per-soldier offensive lethality, analogously defined; here slightly higher than a, so Blue's soldiers are individually more effective, but Blue fields fewer of them); simulate over $t \in [0, 50]$.

Concentration principle (square law): Red wins iff $aR_0^2 > bB_0^2$. So doubling troops quadruples effective firepower, a huge advantage.

Getting Started.

1. Reuse your vector Forward-Euler solver.
2. Write two rate functions: `square_law(y, a, b)` and `linear_law(y, a, b, R0)`.
3. Integrate until one side reaches 0 (check each step); stop early.
4. For each simulation, report who wins, when, and with how many survivors.

Tasks.

1. Implement both models. Plot $R(t), B(t)$ under each law with the baseline parameters on two separate figures. Who wins in each case?
2. **Verify the concentration principle:** Hold B constant at $(B_0, b) = (800, 0.20)$. For Red, find the combination (R_0, a) that gives a tied battle under the square law (i.e. solve $aR_0^2 = bB_0^2$). Simulate three cases: (i) the tie, (ii) Red has $1.1\times$ troops, (iii) Red has $1.1\times$ fighting effectiveness. Which substitution is more decisive?
3. **Divide vs. concentrate:** Red forces are split into four groups of 250 each, engaging Blue ($B_0 = 800$) sequentially. Simulate each engagement; the survivors from each battle carry into the next. Compare total Red survivors with the "concentrated" scenario (all 1000 engaging 800 at once).
4. **Linear law comparison:** Repeat Task 2 under the linear law. Does the $1.1\times$-troops advantage still dominate? (It shouldn't; the linear law doesn't reward concentration.) Note: the linear law decays asymptotically ($R \to 0$ as $t \to \infty$), so extend your simulation to $t \in [0, 500]$ or stop when R drops below a tolerance (e.g. $R < 1$) and report the surviving side's count.
5. **Historical interpretation:** Nelson at Trafalgar broke the enemy fleet into two isolated groups and destroyed them sequentially. Using the square law, simulate (i) 27 British ships vs. 33 combined Franco-Spanish, and (ii) 27 British vs. two separate groups of 16 and 17. Pick your own a, b; comment.
6. **Asymmetric technology (bonus):** Red has twice the firepower ($a = 0.30$) but 30% fewer troops ($R_0 = 700$). Who wins? Under which law?

Deliverables. `lanchester.py`; 5–6 PDF figures; report with a clear statement of the concentration principle and at least one historical example.

Project-Specific Grading Focus.

- You must terminate the simulation correctly when either side reaches zero; continuing past zero is a red flag.
- The concentration-principle verification must compare the R_0 doubling vs. a doubling outcomes explicitly (this is the conceptual heart of the project).
- Comparing the same scenario under both laws must appear in the report.

Project 6: Prisoner's Dilemma (Axelrod's Tournament) ⋆⋆⋆

Background. In 1980, political scientist Robert Axelrod ran a landmark computer tournament: he invited researchers to submit strategies for a repeated Prisoner's Dilemma, then played them against each other. The simple strategy *Tit-for-Tat* ("copy what your opponent did last time") won both Axelrod's original tournament and a second, larger rerun. The result reshaped game theory, evolutionary biology, and economics. This project recreates the experiment. **It is the only project that requires *no* ODEs**: just functions, lists, dictionaries, and loops.

Payoff Matrix. When both players choose C (cooperate) or D (defect) simultaneously:

	C	D
C	(3, 3)	(0, 5)
D	(5, 0)	(1, 1)

Single-round dominant strategy: D (the "paradox"). Repeated play: cooperation can emerge.

Getting Started.

1. Write each *strategy* as a Python function of signature `strategy(my_history, opp_history) -> "C" or "D"`, where `my_history` and `opp_history` are lists of past moves.
2. Write `payoff(move_a, move_b) -> (score_a, score_b)` using a dictionary lookup.
3. Write `play_match(strategy_a, strategy_b, n=200)`: loops `n` rounds, calls both strategies each round, accumulates scores, and returns the two totals.
4. Write `tournament(strategies_dict, n=200)`: runs every strategy against every other (and itself); returns a dictionary of total scores.

Tasks.

1. Implement **six baseline strategies**: `always_cooperate`, `always_defect`, `tit_for_tat`, `tit_for_two_tats`, `grudger` (cooperates until first defection, then always defects), `random`.
2. Implement `payoff`, `play_match`, `tournament`.
3. **Run the tournament**; print the leaderboard and plot a bar chart of total scores.
4. **Duel analysis:** Plot cumulative scores of `tit_for_tat` vs. `always_defect` over 200 rounds; do the same for `tit_for_tat` vs. `tit_for_tat`. Discuss the difference.
5. **Two invented strategies:** Design at least two new strategies of your own (e.g. "cooperate with probability equal to opponent's cooperation rate so far", or "defect every 5th round regardless"). Include them in the tournament. Do they win?
6. **Noise robustness:** Modify `play_match` so that each move is flipped with probability $p = 0.05$ (simulating miscommunication or "trembling hand"). Rerun the tournament. Does `tit_for_tat` still win? Why or why not?
7. **Evolutionary dynamics (bonus):** Start a population of 100 agents with 1 copy each of the 6+ strategies. Each round, every agent plays every other; then the bottom strategy loses one copy and the top gains one. Run 100 generations. Plot population fractions over time.
8. **Report discussion:** Axelrod identified four properties of successful strategies: *Nice* (never defects first), *Retaliatory*, *Forgiving*, *Clear*. Discuss how each of your tested strategies fits (or fails) these criteria.

Deliverables. `strategies.py`, `tournament.py`; leaderboard bar chart, cumulative-score plots, noise comparison; report with Axelrod's four-property analysis.

Project-Specific Grading Focus.

- Strategies must be *first-class functions* passed as arguments, not `if`-statements inside `play_match`. This is the key programming lesson.
- The two self-designed strategies must be genuinely different from the baselines and analysed.
- The report must explain *why* Tit-for-Tat wins (or doesn't, under noise) in terms of Axelrod's four properties.

Project 7: Hawk–Dove Evolutionary Game ⋆⋆⋆⋆

Background. Why don't animals always fight to the death? In 1973, biologist John Maynard Smith modelled two behavioural types: aggressive "Hawks" that escalate conflict, and timid "Doves" that back down. He showed that in equilibrium *neither* dominates. The stable mix is an *Evolutionarily Stable Strategy* (ESS), a concept that has migrated from biology into economics

and political science.

Model. Let V = value of a contested resource, C = cost of injury in a Hawk–Hawk fight, with $C > V$. Payoff matrix (expected payoff to row type):

	H	D
H	$(V-C)/2$	V
D	0	$V/2$

Let x = fraction of Hawks in the population. Fitnesses:

$$f_H(x) = x \cdot \tfrac{V-C}{2} + (1-x)V, \qquad f_D(x) = (1-x) \cdot \tfrac{V}{2}, \qquad \bar{f}(x) = x f_H + (1-x) f_D.$$

Replicator equation (continuous-time evolutionary dynamics):

$$\dot{x} = x\big[f_H(x) - \bar{f}(x)\big] = x(1-x)\big[f_H(x) - f_D(x)\big].$$

When $C > V$, the interior equilibrium $x^* = V/C$ is the ESS.

Baseline parameters: $V = 2$, $C = 6$, so $x^* = 1/3$. Simulate $t \in [0, 30]$.

Getting Started.

1. Write $f_H(x)$, $f_D(x)$ as small helper functions.
2. Write the replicator rate: `def xdot(t, x): return x*(1-x)*(fH(x) - fD(x))`.
3. Use Forward Euler to simulate from several initial $x_0 \in (0, 1)$; overlay trajectories.

Tasks.

1. **Derivation:** In your report, derive the replicator equation and show algebraically that the interior equilibrium is $x^* = V/C$.
2. Simulate $x(t)$ from $x_0 \in \{0.05, 0.3, 0.5, 0.8, 0.95\}$ on one figure. Mark the ESS line.
3. **Equilibrium via root-finding:** Use Newton's method (Lesson 10) on $\dot{x}(x) = 0$ to locate all equilibria numerically. You should find three: $x = 0$, $x = 1$, and $x = V/C$.
4. **Stability analysis:** Linearise $\dot{x}$ around each equilibrium (compute $d\dot{x}/dx$ at each). Which are stable? Verify numerically by perturbing x^* by ± 0.05 and observing the return.
5. **Parameter regimes:** On a (V, C) grid with $V \in [0.5, 5]$, $C \in [0.5, 10]$, colour each cell by the long-run outcome: (all-Hawk, all-Dove, mixed ESS, pathological). Heatmap.
6. **Three-strategy extension (bonus):** Add *Bourgeois*, a strategy that behaves as Hawk when an "owner" of a resource and Dove otherwise. Formulate a 3-species replicator system and show that pure Bourgeois is the ESS (this requires a bit more bookkeeping).

Deliverables. `hawk_dove.py`; trajectory overlay, (V, C) heatmap, stability diagram; report with the replicator derivation.

Project-Specific Grading Focus.

- The replicator derivation must be in the report (this is a conceptual test).
- You must identify *all three* equilibria and classify their stability.
- The (V, C) heatmap must correctly capture the four qualitative regimes.

Project 8: Cournot Duopoly Dynamics ⋆⋆⋆⋆

Background. In 1838, Antoine Augustin Cournot studied two companies selling the same product, each choosing how much to produce. The higher the combined supply, the lower the market price. Cournot's analysis is the earliest known use of Nash equilibrium (110 years before Nash!). Today it underpins antitrust law, telecom-spectrum auctions, and OPEC production-quota negotiations.

Model. Two firms choose outputs $q_1, q_2 \geq 0$. Market price: $P = a - b(q_1 + q_2)$. Each firm has marginal cost c; profits: $\pi_i = (P - c)q_i = [a - b(q_1 + q_2) - c]q_i$.

Best response of firm i to a fixed q_{-i}: set $\partial \pi_i / \partial q_i = 0$ to get

$$q_i^*(q_{-i}) = \max\left(\frac{a - c - bq_{-i}}{2b},\ 0\right).$$

Nash equilibrium: $q_1^* = q_2^* = (a - c)/(3b)$, giving profits $\pi_i^{\text{Nash}} = (a - c)^2/(9b)$.

Parameters: $a = 100$ (demand intercept, the price consumers would pay for the first unit when total supply is zero), $b = 1$ (demand slope, the price drops by \$1 for every extra unit the two firms collectively produce), $c = 20$ (constant marginal cost of producing one more unit, identical for both firms). Nash output: $q^* = (a - c)/(3b) = 80/3 \approx 26.67$ per firm.

Getting Started.

1. Write `best_response(q_other, a, b, c)` returning the analytical best response.
2. **Simultaneous updates:** Start at $(q_1, q_2) = (5, 10)$. Each iteration: $q_1^{\text{new}} = \text{BR}(q_2)$, $q_2^{\text{new}} = \text{BR}(q_1)$. Print the sequence.
3. **Cobweb diagram:** Plot the iteration trajectory $\{(q_1^{(k)}, q_2^{(k)})\}$ on the (q_1, q_2) plane together with both best-response curves. The trajectory should spiral to the Nash equilibrium.

Tasks.

1. Derive the best-response function analytically; show your work in the report.
2. Implement the simultaneous-update dynamics. Verify convergence to $q^* = (a - c)/(3b)$ from three different initial points.
3. **Cobweb plot:** Draw both best-response lines and the trajectory; label the Nash equilibrium.
4. **Profit comparison (Nash vs. collusion):** Under collusion, the two firms act as a single monopolist and jointly maximise $(a - bQ - c)Q$ where $Q = q_1 + q_2$. Find the monopoly total quantity Q_{mon}^* and compare total industry profit with the Nash case. Why does Nash give *lower* combined profit? (This is an instance of the Prisoner's Dilemma; it connects directly to Project 6.)
5. **Sensitivity:** Perturb (a, b, c) one at a time by $\pm 20\%$. Report the effect on the Nash quantity and profits.
6. **Three-firm oligopoly:** Extend the model to three firms; derive the new Nash equilibrium ($q^* = (a - c)/(4b)$). Verify via simulation. *Numerical note:* With three firms the fully synchronous best-response iteration $q_i^{(k+1)} = \text{BR}_i\big(q_{-i}^{(k)}\big)$ does *not* converge; its linearised update has an eigenvalue of -1 and the trajectory enters a period-2 cycle around the Nash point. Use a damped update $q_i^{(k+1)} = \frac{1}{2}\,\text{BR}_i + \frac{1}{2}\,q_i^{(k)}$, or update one firm at a time (asynchronous / Gauss-Seidel). Comment briefly in the report on why the undamped scheme fails.

7. **Stackelberg variant (bonus):** Now assume firm 1 moves first, and firm 2 observes q_1 and responds optimally. Find the Stackelberg equilibrium analytically and simulate. Is firm 1 better off? Is firm 2 better off?
8. **Numerical best-response (bonus):** Instead of the analytical formula, find each firm's best response by *numerically maximising* profit over $q_i \in [0, 100]$ using a golden-section search or Newton's method on $\partial\pi/\partial q$. Verify you get the same Nash equilibrium.

Deliverables. `cournot.py`; cobweb trajectory, profit time-series, Nash vs. collusion bar chart; report with best-response derivation and the Prisoner's-Dilemma connection.

Project-Specific Grading Focus.

- The best-response derivation must be complete and correct.
- The collusion-vs.-Nash comparison must explicitly quote both profits and explain why a "rational" Nash outcome is collectively worse.
- The cobweb diagram must include *both* best-response lines.

Project 9: Lorenz Attractor & The Butterfly Effect ⋆⋆⋆⋆⋆

Background. In 1963, MIT meteorologist Edward Lorenz reduced a convection model to three simple equations and accidentally discovered *chaos*. He found that a perturbation in the fifth decimal place of the initial condition changed the weather forecast entirely within a few simulated days, "the flap of a butterfly's wings in Brazil causing a tornado in Texas." The three equations below are the most iconic in nonlinear dynamics. Solving them teaches sensitivity, numerical integration, and 3-D visualisation in one go.

Model.

$$\begin{aligned} \dot{x} &= \sigma(y - x), \\ \dot{y} &= x(\rho - z) - y, \\ \dot{z} &= xy - \beta z. \end{aligned}$$

Here $x(t)$ measures the intensity of convection, $y(t)$ the horizontal temperature difference, and $z(t)$ the vertical temperature profile distortion. Classical parameters ("strange attractor" regime):

- $\sigma = 10$: *Prandtl number*, the ratio of fluid viscosity to thermal diffusivity (roughly how fast momentum diffuses compared with heat).
- $\rho = 28$: *Rayleigh number* (normalised), a measure of the temperature-gradient forcing that drives convection; chaos appears above the critical value $\rho \approx 24.74$.
- $\beta = 8/3$: *geometric aspect ratio* of the convection roll (width vs. height); Lorenz chose this value from the rolled-up-cell geometry of his original atmospheric model.

Initial condition: $(x_0, y_0, z_0) = (1, 1, 1)$. Simulate $t \in [0, 40]$ with $\Delta t = 0.01$.

Getting Started.

1. Reuse your vector Forward-Euler solver with a 3-D state.
2. Write the Lorenz rate function returning a length-3 NumPy array.
3. Plot $x(t)$, $y(t)$, $z(t)$ each as separate time-series panels. They should look "noisy" but actually be deterministic.
4. Import `mpl_toolkits.mplot3d`; plot the trajectory in 3-D. You should see the famous butterfly.

Tasks.

1. Solve the Lorenz system; plot three time-series panels and the 3-D attractor.
2. **Three 2-D projections** (x-y, x-z, y-z) in one figure.
3. **Butterfly effect:** Perturb the initial x_0 by $\delta = 10^{-5}$; integrate both trajectories. Plot $|\Delta(t)| = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2}$ on a semi-log y-axis. In the initial growth phase, $|\Delta| \sim \delta\, e^{\lambda t}$; fit a line to estimate the *Lyapunov exponent* λ. The published value is $\lambda \approx 0.9$.
4. **Time-step sensitivity:** Rerun with $\Delta t \in \{0.1, 0.02, 0.01, 0.002\}$. What happens to Forward Euler at large Δt? Which Δt gives a stable simulation for $t \in [0, 40]$?
5. **Solver comparison:** Implement Runge–Kutta 4 (RK4) for vector ODEs. Compare the Lyapunov estimate from Forward Euler and RK4. Which is more trustworthy, and why?
6. **Bifurcation:** Vary ρ from 0.5 to 30 in steps of 0.5. For each ρ, simulate to steady state and plot z-extrema vs. ρ (a bifurcation diagram). Identify the transition from stable fixed point $\rightarrow$ periodic orbit $\rightarrow$ chaos.
7. **Non-chaotic regimes (bonus):** Find a ρ where the system settles to a stable equilibrium and one where it forms a periodic limit cycle. Include phase portraits for each.
8. **Fractal dimension (bonus, truly hard):** Estimate the correlation dimension of the strange attractor via box-counting on the point cloud. The published value is $D \approx 2.06$.

Deliverables. `lorenz.py`, `rk4.py`; 3-D attractor, three 2-D projections, divergence plot with Lyapunov fit, bifurcation diagram; report with the physical story (convection), your Lyapunov estimate, and a discussion of why weather forecasting has fundamental limits.

Project-Specific Grading Focus.

- The Lyapunov fit must be quantitative and compared with the published value.
- RK4 vs. Euler comparison must be included; Forward Euler is dangerously inaccurate for chaotic systems and a good student must recognise this.
- Because of the difficulty, up to **8 bonus points** are available; partial credit is generous.